



TRABAJO DE FINAL DE GRADO

GRADO EN INTELIGENCIA ROBÓTICA

Estudio e implementación del control de péndulo invertido sobre carro en un sistema de tiempo real

Estudiante:
Miguel Porcar Vicent

Tutor:
Ignacio Peñarrocha Alós

Fecha de presentación: Junio de 2026
Curso académico: 2025/2026

Agradecimientos:

Palabras clave:

CASTELLÓN, a XX de XXXXXXX de 2025

Todos los derechos reservados

ÍNDICE GENERAL

1	Introducción	9
1.1	Motivaciones	9
1.2	Objetivos	9
1.3	Planificación	9
1.4	Estimación de costes	9
2	Análisis	11
2.1	Requisitos	11
2.1.1	Requisitos funcionales	11
2.1.2	Requisitos no funcionales	12
2.2	Arquitectura del sistema	12
2.2.1	A1. Arquitectura de control	12
2.2.2	A2. Arquitectura de la simulación	13
2.2.3	A3. Arquitectura del sistema real	14
2.3	Modelado matemático	15
2.3.1	Dinámica del sistema	15
2.3.2	Linealización	15
2.3.3	Representación en el espacio de estados	15
2.3.4	Diseño del controlador	15
3	Implementación	17
3.1	Simulación	17
3.2	Sistema físico	17

4	Resultados	19
4.1	Resultados en simulación	19
4.2	Resultados en sistema físico	19
5	Conclusiones y posible trabajo futuro	21
	Bibliografía	23
6	Apéndices	25

ÍNDICE DE FIGURAS

2.1	Arquitectura de control	12
2.2	Arquitectura de la simulación	13
2.3	Arquitectura del sistema real	14

ÍNDICE DE TABLAS

1. Introducción

1.1 Motivaciones

En caso de que haya motivo(s) concreto(s) para la realización del proyecto.

1.2 Objetivos

Objetivos que se espera alcanzar al inicio del proyecto.

1.3 Planificación

Lista de tareas, relación entre ellas y duración estimada Posibles desviaciones de la planificación Diagrama de Gantt

1.4 Estimación de costes

Estimación del coste económico que supondría la adquisición del material necesario (hardware y software) para la realización del proyecto, los salarios involucrados y otros aspectos como alquileres de espacios y energía.

2. Análisis

2.1 Requisitos

Para dotar de rigor metodológico al proyecto, los requisitos se han dividido en funcionales (comportamiento del sistema) y no funcionales (restricciones y criterios de calidad). Esta clasificación permitirá, en capítulos posteriores, validar el grado de consecución de los objetivos planteados.

2.1.1 Requisitos funcionales

Los requisitos funcionales describen las acciones, funciones y comportamientos específicos que el sistema debe realizar para cumplir su propósito. Para el control de un péndulo invertido sobre carro se definen:

- **RF1. Modelado y simulación:** El sistema debe integrarse en un entorno simulación (Matlab/Simulink) para el desarrollo de algoritmos de control como paso previo a su implementación en hardware real. Se requiere el modelado de la dinámica no lineal y la introducción de los parámetros (masas, inercias, rozamientos...) que caracterizan el sistema real.
- **RF2. Visualización y telemetría en ROS:** Se requiere establecer un puente de comunicación (ROS Toolbox) que publique el estado del sistema (*joint states*) para su visualización en tiempo real mediante un modelo URDF en Rviz.
- **RF3. Control de inyección de energía (Swing-up):** El control debe ser capaz de balancear el péndulo desde su posición estable ($q = \pi rad$) hasta su extremo opuesto ($q = 0 rad$) mediante técnicas de inyección de energía.
- **RF4. Control de estabilización (Realimentación del estado):** El sistema debe conmutar automáticamente a un control por realimentación del estado para mantener el péndulo en la vertical y posicionar el carro en un punto del riel.
- **RF5. Adquisición de señales:** El firmware debe procesar las señales de los encoders en cuadratura y ejecutar la ley de control con una frecuencia de muestreo suficiente para garantizar la estabilidad del sistema inestable.

2.1.2 Requisitos no funcionales

Los requisitos no funcionales, describen como debe comportarse el sistema y son esenciales para determinar su calidad. Para el caso en concreto:

- **RNF1. Determinismo y tiempo real estricto:** El bucle de control debe ejecutarse dentro de una rutina de interrupción (ISR) de prioridad máxima en la DSP, garantizando un periodo de muestreo constante y un *jitter* despreciable.
- **RNF2. Robustez y márgenes de estabilidad:** El sistema debe ser capaz de rechazar perturbaciones externas moderadas y ser tolerante a las discrepancias producidas por las dinámicas no modeladas (fricciones no lineales y holguras mecánicas).
- **RNF3. Seguridad y gestión de fallos:**
 - *Mecánica:* El software debe implementar una parada de emergencia si el carro excede los límites de seguridad del riel.
 - *Eléctrica:* Se debe asegurar el aislamiento galvánico mediante optoacopladores entre la etapa de potencia (24V) y la etapa de control (3.3V).
- **RNF4. Abstracción de hardware (HAL):** Se debe emplear las librerías de bajo nivel ofrecidas por el fabricante de la DSP para asegurar su correcta programación, portabilidad y estandarización del código.

2.2 Arquitectura del sistema

El desarrollo de un sistema robótico de equilibrio inestable requiere una visión multidisciplinar. Por ello, la arquitectura de este proyecto se ha estructurado en tres niveles de abstracción complementarios: el diseño lógico del control, el entorno de validación virtual y la implementación física del hardware.

2.2.1 A1. Arquitectura de control

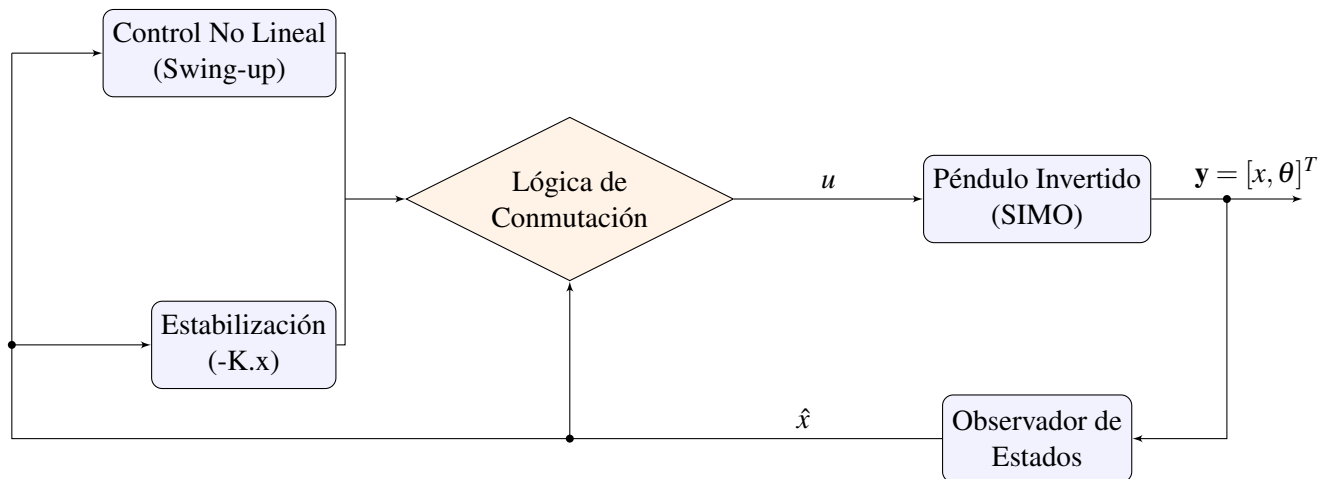


Figura 2.1: Arquitectura de control

La arquitectura de control propuesta en la Figura 2.1 se aplica sobre un sistema subactuado de tipo SIMO (Single Input Multiple Output). Para la etapa de realimentación, se implementa un observador del estado por inversión. Mediante este enfoque, los estados del sistema se obtienen

exclusivamente a partir de las mediciones de salida, dado que, en el caso de sistemas robóticos, el estado coincide con las salidas ($q, \dot{q}, \ddot{q}$).

Finalmente, lógica de conmutación atiende a un umbral angular. Este bloque de decisión monitoriza de forma continua la posición angular estimada del péndulo ($\hat{\theta}$). Si el péndulo ingresa dentro de una vecindad acotada en torno a la vertical absoluta (*el punto de linealización del sistema*), la lógica conmuta al control por realimentación del estado. En caso contrario, el sistema mantiene activo el controlador de swing-up para elevar el mecanismo. Este comportamiento híbrido se formaliza en el siguiente pseudocódigo:

Algoritmo 1 Lógica de Conmutación

Require: Vector de estados estimado $\hat{x} = [\hat{x}_c, \dot{\hat{x}}_c, \hat{\theta}, \dot{\hat{\theta}}]^T$, umbral angular θ_{th}

Ensure: Señal de control u a aplicar a la planta

```

1: loop
2:    $\hat{\theta} \leftarrow \hat{x}(3)$                                 ▷ Obtener posición angular actual
3:   if  $|\hat{\theta}| \leq \theta_{th}$  then
4:      $u \leftarrow -K\hat{x}$                                 ▷ Control de posición
5:   else
6:      $u \leftarrow f_{swing}(\hat{x})$                         ▷ Control de energía
7:   end if
8:
9: end loop

```

2.2.2 A2. Arquitectura de la simulación

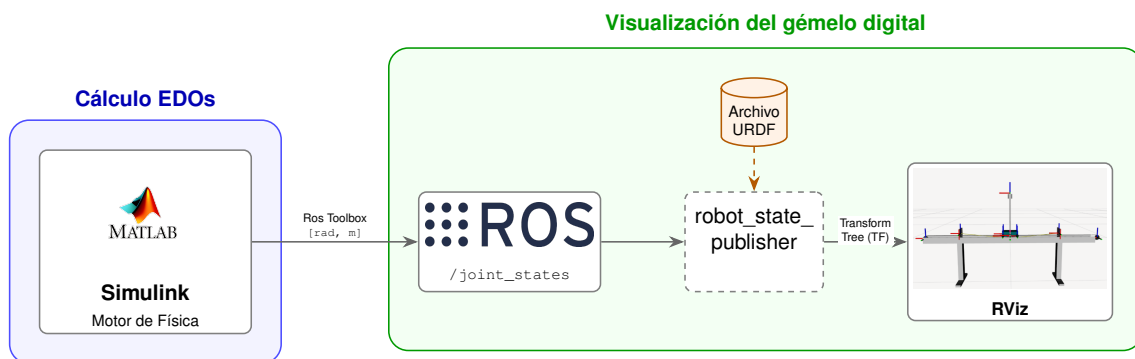


Figura 2.2: Arquitectura de la simulación

La Figura 2.2 detalla la arquitectura de software diseñada para la simulación del sistema. A continuación se desglosa cada elemento del esquema para conocer su función y como se conecta con el resto de la arquitectura:

- **Cálculo de EDOs (Motor de Física):** Implementado en el entorno MATLAB/Simulink, este bloque se encarga de resolver las Ecuaciones Diferenciales Ordinarias (EDO) que describen la dinámica del sistema. Simulink actúa como el motor de física, calculando en cada paso de tiempo los estados del robot (posiciones en radianes y metros). Estos datos se transmiten al ecosistema ROS mediante el ROS Toolbox, publicando la información en el tópico `/joint_states`.

- **Visualización, gemelo digital:** Este bloque recibe la información cinemática para representar el comportamiento del sistema en tiempo real. Está totalmente integrado en el entorno de ROS y se compone de varias piezas:

- **Archivo URDF (Unified Robot Description Format):** Es un archivo XML que describe como es el robot o mecanismo que se quiere simular. Se especifican las visuales, las restricciones cinemáticas y los marcos de referencia de las articulaciones.
- **Robot State Publisher:** Es el nodo de ROS que permite representar el movimiento del robot. Se suscribe al tópico `/joint_states` y actualiza el árbol de transformadas de nuestro archivo URDF.
- **RViz:** La última pieza de la arquitectura software, es la interfaz que permite visualizar de forma gráfica e intuitiva todo el flujo de datos en un modelo 3D del mecanismo.

2.2.3 A3. Arquitectura del sistema real

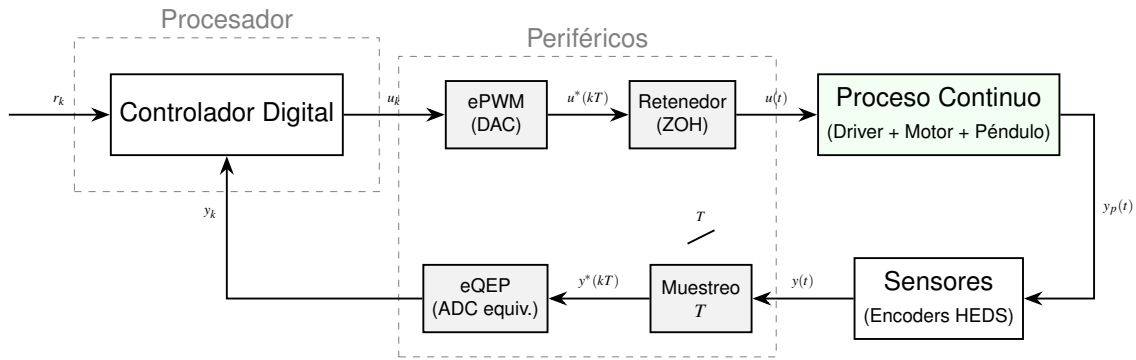


Figura 2.3: Arquitectura del sistema real

La Figura 2.3 presenta de forma esquemática la implementación del bucle de control en un microcontrolador.

Esta estructura permite observar la interacción entre el entorno digital (discreto) y el proceso físico (continuo). El núcleo del sistema es el **Controlador Digital**. Este procesador se encarga de ejecutar el algoritmo de control que procesa la señal de error entre la referencia (r_k) y la realimentación (y_k).

El concepto fundamental que rige esta arquitectura es el **periodo de muestreo (T)**. Este parámetro determina la periodicidad de la rutina de control, permitiendo sincronizar las etapas de medida, cálculo y actuación sobre la planta de manera precisa. Durante el desarrollo del proyecto se determinará el periodo de muestreo atendiendo a factores como la dinámica del sistema en bucle cerrado o la frecuencia de la señal PWM.

Finalmente la interacción entre el mundo digital y el mundo físico es posible mediante el uso de periféricos que nos permiten medir el estado del sistema o modificarlo. Diferenciamos en dos clases de periféricos:

- **Actuadores:** La señal de control digital (u_k) se convierte en una señal analógica o cuasi-analógica mediante el módulo *ePWM* (DAC, convertidor digital-analógico). Posteriormente, un bloque *Retenedor de orden cero* (ZOH) mantiene la señal constante durante el intervalo de muestreo, transformándola en la señal continua $u(t)$ que excita al Proceso Continuo (compuesto por el driver, el motor y el péndulo).
- **Sensores:** La respuesta del sistema ($y_p(t)$) es captada por los Sensores (en este caso, encoders). La señal $y(t)$ es procesada por un bloque de muestreo, gobernado por el periodo T ,

y enviada al periférico *eQEP*. Este último actúa como un equivalente a un ADC (convertidor analógico-digital), transformando los pulsos del encoder en una señal digital y_k interpretable por el controlador.

2.3 Modelado matemático

2.3.1 Dinámica del sistema

Aquí el desarrollo de las ecuaciones de movimiento y el encapsulado en las matrices M, G, C y B.

2.3.2 Linealización

Explicar el porque se debe linealizar el sistema y como se hace.

2.3.3 Representación en el espacio de estados

Utilizar la técnica de linealización para obtener los estados del sistema a partir de las ecuaciones de movimiento. Matrices A, B, C, D.

2.3.4 Diseño del controlador

Control de energía, Swing up

Explicar las bases matemáticas de la inyección de energía en un sistema para levantarlo a una posición. Comentar como en el libro underactuated utilizan PFL para el swing up de péndulo invertido sobre carro.

Control de posición

Explicar como se ha obtenido un controlador por realimentación del estado mediante el uso del algoritmo LQR, comentar la relación entre Q y R (son matrices que dan más peso a la convergencia o al uso del actuador)

3. Implementación

Explicación de las actividades realizadas durante el desarrollo del proyecto. Se puede exponer en orden cronológico o por grupos de tareas. Hay que indicar los hitos alcanzados.

En esta sección solamente hay que aportar la información técnica estrictamente necesaria para comprender el trabajo realizado. Esta información debe, además, estar presentada de forma que sea inteligible por cualquiera no familiarizado con los aspectos técnicos del trabajo realizado. La información técnica puede ubicarse en los apéndices para ser consultada si se desea.

3.1 Simulación

1) Uso de parámetros proporcionados por el fabricante (rozamiento, inercias, etc.) para modelado del sistema en Matlab/Simulink.

2) Conexión exitosa entre ROS y Matlab mediante la herramienta ROS Toolbox, hay que comentar como se tubo que cambiar el DDS al cyclone para funcionar, la lógica de publicador subscriber, el diseño de un archivo URDF para la visualización de la unidad mecánica desde RViz. (Posibles vías de estudio como la migración de la simulación completamente a ROS)

3) Explicar también como mediante simulación se pudo obtener las ganancias necesarias para poder hacer control swin-up con PFL. Muy importante poner los diagramas de bloques de nuestro simulink (El de publicación de tópicos, el de dinámica del sistema y el de conmutación de control (Swing-Up to LQR))

4) Finalmente explicar como se acabo implementando una simulación discreta del sistema (uso de bucle for y mida de paso con el método de Euler o1). Sirve como puente a la implementación de algoritmos de control en microcontroladores.

3.2 Sistema físico

1) Selección de microcontrolador, Pte H y encoders. Explicar un poco la tecnología detrás de cada componente (Ejemplo: Sabemos que los encoders son HEDS-9140-A00, también sabemos que el PTE H tiene aislamiento galvánico mediante optoacopladores).

2)Firmware, comentar como se programa una DSP de texas instruments mediante el uso del IDE CCS y la capa de HAL C2000-ware. Utilizar el datashet del fabricante para justificar la configuración de ciertos registros. Módulos Epwm y Eqp muy interesante ver como liberan a la CPU de estas tareas.

3)Explicar como se programo la DSP con timers para ejecutar el bucel de control en una rutina de interrupción periódica (implica determinismo, nada del loop de Arduino)

4)(Lo que hay que acabar de hacer este més) Razonar experimentos de identificación de parámetros del sistema. La justificación es que los parámetros del fabricante no sabemos si son del todo ciertos al ser una máquina de hace 25 años. Algunos experimentos: Conversión de voltios a fuerza horizontal (encontrar la constante que relaciona ambas magnitudes), Identificación del rozamiento horizontal del carro, identificación del rozamiento y la inercia del péndulo.

5)Simular de nuevo con los parámetros obtenidos mediante identificación.

4. Resultados

- 4.1 Resultados en simulación
- 4.2 Resultados en sistema físico

5. Conclusiones y posible trabajo futuro

Trabajo a futuro → Implementación en HAL, uso de algoritmos no basados en modelo (Reinforcement Learning)

Bibliografía

- [1] Russ Tedrake. *Underactuated Robotics: Algorithms for Planning and Control*. Disponible online en <http://underactuated.mit.edu/>. MIT Press, 2023.
- [2] *TMS320F28335 Digital Signal Controller (DSC) Data Sheet*. manual. Literature Number: SPRS439N. Texas Instruments. 2024.
- [3] MathWorks. *ROS Toolbox Documentation*. misc. 2026. URL: <https://es.mathworks.com/products/ros.html>.

6. Apéndices

Colocar aquí cualquier tipo de información susceptible de ser consultada para la correcta comprensión del documento y el proyecto que describe.